

VisionInspect AI (Milestone-2)

MongoDB Integration — Backend Architecture Report

Om Shakthi Vemuganti

1. Executive Summary

The VisionInspect AI backend is a Flask web service handling image inspections. Two databases work together: a relational store (SQLite in development, PostgreSQL planned) for user and inspection metadata, and a MongoDB Atlas cloud database for AI inference results — anomaly scores, masks, and detected defects.

Every image upload goes through the same path: the image is saved, an `inspection_results` row is created, and the AI output for that same inspection is written to MongoDB's `inference_results` collection, linked by a shared `inspection_id`.

Status: MongoDB integration is functionally complete on the backend. Model output is currently demo/placeholder values — the real PatchCore model is the next hookup, alongside migrating SQLite to PostgreSQL.

2. System Components

- Flask backend (Python) — hosts REST APIs (`/upload`, `/inspection`, etc.) and ties everything together
- Relational DB (SQLite → PostgreSQL) — `users` table for credentials; `inspection_results` for each inspection's metadata
- MongoDB Atlas — managed cloud database; created cluster + `visioninspect_ai` database + `inference_results` collection
- MongoDB Compass — desktop GUI used to visually confirm documents were inserted correctly
- AI Inference (PatchCore) — anomaly detection model; demo values used for now, real model is next step

3. Relational (SQL) Schema

Two tables. users holds credentials; inspection_results holds one row per uploaded image, linked to the user who uploaded it.

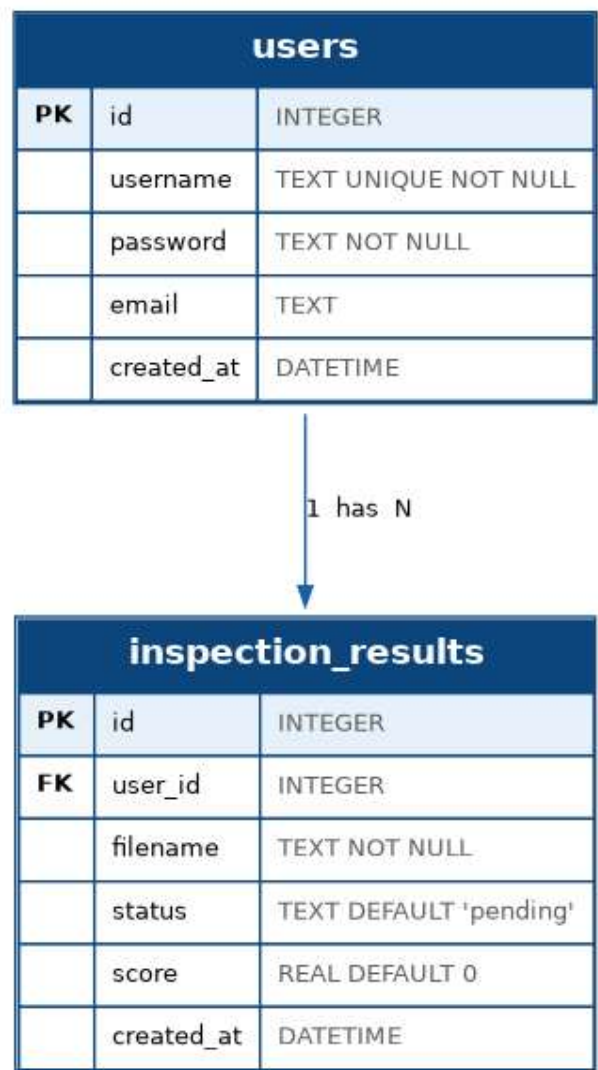


Figure 1 — SQL ER diagram: users (1) → inspection_results (N)

Note: user_id is not enforced as a foreign key by SQLite by default — this constraint should be added explicitly once migrated to PostgreSQL.

4. MongoDB Document Schema

One collection, inference_results — one document per inspection, embedding the variable-length defect list instead of a join. Linked to the SQL side by inspection_id, which should carry a unique index to enforce a strict one-to-one relationship.

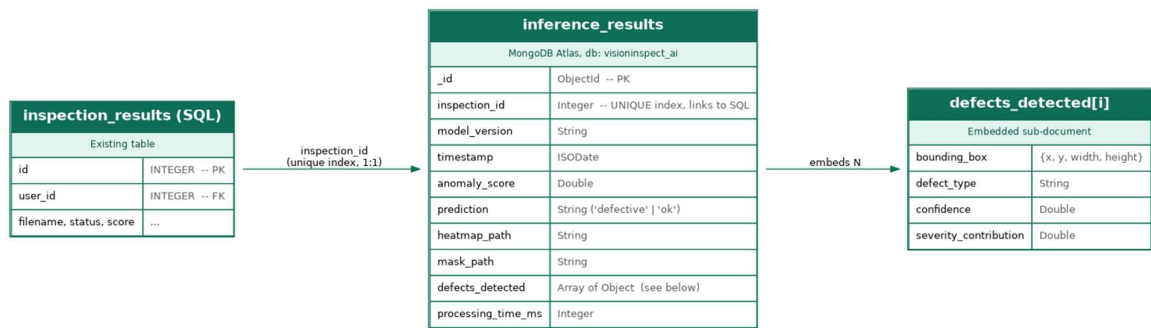


Figure 2 — MongoDB document schema, unique-indexed on `inspection_id`

Sample document

```
{
  "_id": ObjectId("60d5..."),
  "inspection_id": 1,
  "model_version": "patchcore-demo",
  "timestamp": ISODate("2026-07-20T12:00:01Z"),
  "anomaly_score": 0.85,
  "prediction": "defective",
  "heatmap_path": "/uploads/heatmaps/1_heatmap.png",
  "mask_path": "/uploads/masks/1_mask.png",
  "defects_detected": [ { "bounding_box": { "x": 120, "y": 84, "width": 40, "height": 22 },
    "defect_type": "scratch", "confidence": 0.91,
    "severity_contribution": 27.3 } ],
  "processing_time_ms": 300
}
```

5. Data Flow

Runtime sequence for one inspection, from upload to retrieving the AI result:

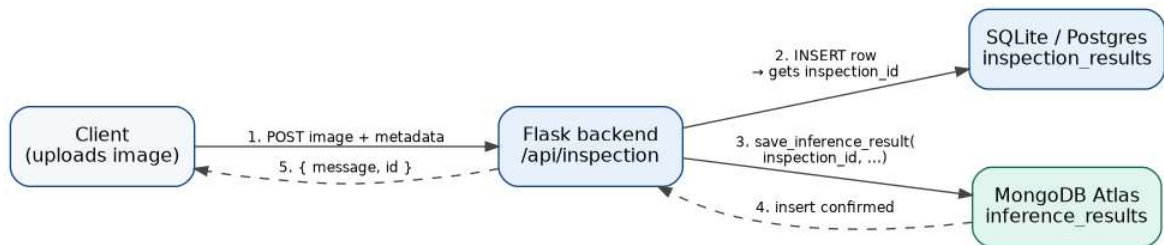


Figure 3 — Request flow: Client → Flask → SQL → MongoDB

Step	Action
1	Client POSTs image + metadata to /api/inspection
2	Flask inserts a row into inspection_results (SQL) → gets inspection_id
3	Flask calls save_inference_result(inspection_id, ...) → writes to MongoDB
4	MongoDB confirms the insert
5	Flask returns { message, id } to the client
—	Later: GET /api/inspection/<id>/raw → find_one({inspection_id}) → returns the AI document

6. Implementation — Files Created / Modified

6.1 database/mongo_connection.py (new)

Reads MONGO_URI and MONGO_DB_NAME from environment and returns a MongoClient database handle.

```
from pymongo import MongoClient
import os

client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("MONGO_DB_NAME")]
```

6.2 services/inference_log_service.py (new)

Two functions: save_inference_result() inserts a document with a UTC timestamp and returns the inserted _id; get_inference_result(inspection_id) queries by that ID and returns the document (with _id cast to string).

```
def save_inference_result(...):
    db = get_mongo_db()
    document = {
        "inspection_id": inspection_id,
        "model_version": model_version,
        "timestamp": datetime.utcnow(),
        "anomaly_score": anomaly_score,
        "prediction": prediction,
        "heatmap_path": heatmap_path,
        "mask_path": mask_path,
        "defects_detected": defects_detected,
        "processing_time_ms": processing_time_ms,
    }
    result = db[COLLECTION].insert_one(document)
    return str(result.inserted_id)
```

6.3 routes/inspection.py (modified)

After the existing SQLite insert + commit, added a call to save_inference_result() with the same inspection_id. Also added a new read route.

```
save_inference_result(
    inspection_id=inspection_id,
    model_version="patchcore-demo",
    anomaly_score=score,
    prediction=status,
    heatmap_path="uploads/demo_heatmap.png",
    mask_path="uploads/demo_mask.png",
    defects_detected=[ { ... } ],
    processing_time_ms=300
)

@inspection_bp.route("/<int:inspection_id>/raw", methods=["GET"])
def get_raw(inspection_id):
    return get_inference_result(inspection_id)
```

6.4 .env (new, not committed)

```
MONGO_URI=mongodb+srv://<user>:<password>@cluster0.mongodb.net
MONGO_DB_NAME=visioninspect_ai
```

```
SECRET_KEY=...
```

6.5 Test scripts (backend/)

Script	Purpose
test_mongo.py	Connects to Atlas, prints "MongoDB Connected"
insert_test.py	Calls save_inference_result() with a sample payload, prints inserted ID
test_read_mongo.py	Calls get_inference_result(), confirms reading works
test_api_inspection.py	POSTs to /api/inspection end-to-end, prints the HTTP response

7. MongoDB Atlas & Compass

- Atlas cluster: free shared-tier cluster created; database visioninspect_ai; the inference_results collection is created automatically on first insert. Atlas manages backups and encryption.
- Compass: connected to the same cluster URI to visually browse visioninspect_ai → inference_results and confirm inserted documents and field values.

8. Indexing & Data Integrity

Recommended: a unique index on inspection_id so each inspection has at most one AI result and lookups stay fast:

```
db.inference_results.createIndex({ "inspection_id": 1 }, { unique: true });
```

- SQL side: users.username should be UNIQUE.
- Mongo side: _id is unique by default; inspection_id enforced unique via the index above.
- No relational FK exists in Mongo — the inspection_id link is conceptual/application-enforced, not database-enforced across the two systems.

9. Security & Deployment Notes

- Credentials (Mongo URI, DB password, Flask SECRET_KEY) are kept in .env, never in code — loaded via python-dotenv.
- Atlas network access: development currently allows all IPs (0.0.0.0/0) — must be restricted to known IPs before production.
- Atlas encrypts traffic via TLS by default.
- Flask's built-in dev server should not be used in production — plan to move to Gunicorn/uWSGI behind nginx.
- SQLite → PostgreSQL migration: use ALTER TABLE for simple additions in dev, but SQLite can't add PRIMARY KEY/UNIQUE columns via ALTER, so some changes need table recreation. PostgreSQL is more flexible here.

10. Status & Next Steps

Item	Status
MongoDB Atlas cluster + database	Done
mongo_connection.py + inference_log_service.py	Done
routes/inspection.py wired to save/read inference results	Done
Compass verification of inserted documents	Done
Test scripts (connection, insert, read, API)	Done
Unique index on inspection_id	Done
Real PatchCore model output (replacing demo values)	Next step
SQLite → PostgreSQL migration	Next step
Frontend using /api/inspection/<id>/raw	Next step
Production WSGI server (Gunicorn/nginx)	Next step

Bottom line: the MongoDB half of the architecture is wired end-to-end and verified — every new inspection triggers a Mongo write, and the raw AI output is retrievable via the new route. What's left is swapping demo values for the real model and moving the relational side to PostgreSQL.